Jamii Savings Bank

Integration Platform

Technical Documentation

Simon Gathu

October 2026

Built with WSO2 Micro Integrator, WSO2 API Manager, Jenkins CI/CD, and RabbitMQ

Contents

1 Architecture Decisions and Trade-offs **2**
1.1 API Gateway and Integration Layer Separation . . . 2
1.2 SOAP-to-REST Transformation at the Integration Layer . . . 2
1.3 Atomic CAR Deployment . . . 2
1.4 Docker Compose for Environment Parity . . . 2

2 Part 1 - Micro Integrator Services **2**
2.1 AccountBalance API . . . 2
2.2 CustomerProxy API . . . 2
2.3 LoanEligibility API . . . 3

3 Part 2 - API Manager **3**
3.1 Published APIs . . . 3
3.2 Throttling Tier Rationale . . . 3
3.3 Custom Gateway Mediation Policy . . . 4
3.4 API Product . . . 4
3.5 Developer Portal Subscription Flow . . . 4

4 Part 3 - Jenkins CI/CD Pipeline **4**
4.1 Pipeline Stages . . . 4
4.2 Failure Handling . . . 4
4.3 Environment Separation . . . 4

5 Bonus A - Dashboard Aggregation API **5**
5.1 Endpoint . . . 5
5.2 The Parallel Design Problem and Solution . . . 5
5.3 Call Flow . . . 5
5.4 Income Calculation . . . 5
5.5 Partial Data Decision . . . 5
5.6 Account to Customer Mapping . . . 6

6 Bonus B - RabbitMQ Message Queue Integration **6**
6.1 Overview . . . 6
6.2 Queues . . . 6
6.3 Message Flow . . . 6
6.4 How This Differs From Request/Response APIs . . . 7

7 Assumptions **7**

8 What I Would Do Differently With More Time **7**

9 How to Run Locally **7**
9.1 Test APIs Directly Against MI . . . 8
9.2 Portals . . . 8

Architecture Decisions and Trade-offs

API Gateway and Integration Layer Separation

APIM handles OAuth2 token validation, throttling, subscription management, and developer portal exposure. MI handles backend orchestration, protocol transformation, data mapping, and error normalization. Each layer has a single responsibility, meaning backend changes never touch APIM and policy changes never touch MI.

SOAP-to-REST Transformation at the Integration Layer

LoanEligibility accepts JSON, transforms it into a SOAP envelope inside MI, calls the calculation backend, then returns clean JSON to the client. The client has no awareness that SOAP is involved. This keeps the gateway policy-clean and makes the transformation independently testable directly against MI.

Atomic CAR Deployment

All three APIs are packaged into a single CAR artifact. Every deployment is atomic, either all APIs deploy or none do, enforcing version consistency across the platform. A failure in one API rolls back all three.

Docker Compose for Environment Parity

MySQL, WSO2 MI, WSO2 APIM, and RabbitMQ run containerized using the same images that would run in production. The Jenkins pipeline tears down and rebuilds the stack on every run, ensuring clean-state testing.

Part 1 - Micro Integrator Services

AccountBalance API

Endpoint: `GET /accounts/{accountNumber}/balance`

Queries MySQL via a WSO2 Data Service using parameterized queries. Returns 404 for unknown accounts and a distinct 500 response for database-level failures. The response envelope is:

```
{
  "accountNumber": "1001001001",
  "status": "ACTIVE",
  "balance": { "amount": 50000.00, "currency": "KES" },
  "requestId": "uuid",
  "timestamp": "epoch-ms"
}
```

CustomerProxy API

Endpoint: `GET /customers/{customerId}`

Proxies JSONPlaceholder (jsonplaceholder.typicode.com) which was chosen as it provides realistic structured user data mirroring what a real CRM would return, with no authentication setup required. The proxy injects a correlation ID header, strips internal headers, configures a 30-second timeout, and has a fault sequence for backend unavailability that returns a consistent error shape.

Technical note: JSONPlaceholder's CDN returns zstd-compressed responses which WSO2 MI cannot decompress. The outbound request explicitly sets `Accept-Encoding: gzip, deflate` to force a compatible encoding.

LoanEligibility API

Endpoint: `POST /loans/eligibility`

Accepts REST/JSON, transforms to a SOAP envelope, calls the public calculator at `www.dneonline.com/calculator.asmx`, and maps the XML response to clean JSON. The SOAP Fault is mapped to a sane REST error response with no raw XML leaked to the client.

Request/Response mapping:

- `monthlyIncome` maps to `intA`
- `loanAmount` maps to `intB`
- `SubtractResult` (income minus loan) becomes `eligibilityScore`
- Positive score means eligible; negative means not eligible

Part 2 - API Manager

Published APIs

API	Context	Tier	Security
AccountBalanceAPI	/accounts	Unlimited	OAuth2
CustomerProxyAPI	/customers	Unlimited	OAuth2
LoanEligibilityAPI	/loans	Gold	OAuth2

Throttling Tier Rationale

LoanEligibilityAPI uses the Gold tier because loan eligibility checks are computationally expensive SOAP operations with an external dependency. Rate limiting this API prevents abuse and protects the downstream calculator service. AccountBalanceAPI and CustomerProxyAPI use Unlimited as balance lookups and customer reads are expected to support high-frequency polling from internal systems.

Note: Gold is a subscription-level tier. The operation-level throttling policy uses Unlimited as Gold does not exist at the resource level in this APIM instance.

Custom Gateway Mediation Policy

AccountBalanceAPI has a custom gateway policy that injects an `X-Bank-Name: JamiiSavings` header on every request. This allows downstream systems and audit logs to identify the originating bank without inspecting the request body. Policy defined at `apis/AccountBalanceAPI/Poli`

API Product

JamiiBankingProduct bundles all three APIs under a single subscription at context `/jamii-banking`. A developer subscribing to the product gets access to all three APIs with one set of credentials, simplifying onboarding.

Developer Portal Subscription Flow

1. Go to `https://localhost:9443/devportal` and log in
2. Click Applications, then Add New Application
3. Go to the API or JamiiBankingProduct, click Subscribe, select your application
4. Go to Applications, your app, Production Keys, click Generate Keys
5. Use the Consumer Key and Secret to generate a Bearer token

Part 3 - Jenkins CI/CD Pipeline

Pipeline Stages

Stage	Description	On Failure
Checkout	Clone repository	Stop pipeline
Build MI Artifacts	mvn clean install creates CAR	Stop pipeline
Start Infrastructure	docker compose up	Stop pipeline
Deploy to MI	CAR deployed, APIs verified	Stop pipeline
Run MI Tests	All three APIs tested	Stop pipeline
Deploy APIs to APIM	All three definitions imported	Stop pipeline
Deploy to Staging	develop branch (placeholder)	Stop pipeline
Deploy to Production	production branch (placeholder)	Stop pipeline

Failure Handling

The pipeline stops immediately if any stage fails. There is no partial deployment. All three APIs deploy together as a single CAR artifact or not at all.

Environment Separation

Dev is fully wired and runs locally via Docker Compose. Staging and Production stages exist in the Jenkinsfile as placeholders demonstrating the promotion model. When a dedicated server is available they are activated by pointing each stage at the correct APIM and MI URLs.

Bonus A - Dashboard Aggregation API

Endpoint

`GET /dashboard/1.0.0/{customerId}/{accountNumber}?loanAmount=20000`

The Parallel Design Problem and Solution

The assignment required calling two or more backends in parallel. The challenge was that the original approach only took `customerId` as input, which meant AccountBalance could not be called without first knowing the account number, making true parallel calls impossible.

The solution was to accept both `customerId` and `accountNumber` as path parameters upfront. With both inputs known at the start of the request, CustomerProxy and AccountBalance have everything they need to fire independently without waiting for each other. This enables the parallel pattern for the first two calls.

LoanEligibility remains sequential because it depends on the balance returned by AccountBalance. You cannot calculate eligibility without knowing the balance first.

Call Flow

```
Request: GET /dashboard/1.0.0/{customerId}/{accountNumber}?
   loanAmount=20000
         |
         +-- CustomerProxy (customerId) -------+
         |                                      +-- both fire
            with known inputs
         +-- AccountBalance (accountNumber) ---+
                   |
                   | balance returned
                   |
         LoanEligibility (monthlyIncome = balance * 5,
            loanAmount from query)
                   |
             Final merged response
```

Income Calculation

Instead of asking the caller to provide monthly income, the account balance is used as a proxy. Balance multiplied by 5 becomes `monthlyIncome` passed to the eligibility engine. This reflects a real banking heuristic where borrowing capacity is proportional to available funds. The `loanAmount` is passed as a query parameter so the caller specifies exactly how much they are requesting.

Partial Data Decision

If the account is not found the API still returns the customer profile. The eligibility section is flagged as `eligible: false` with reason `No transaction history found`.

A customer exists before an account exists and denying profile data because an account is missing would be the wrong behavior for a dashboard.

Account to Customer Mapping

customerId	accountNumber	Name
1	1001001001	John Kamau (ACTIVE)
2	1001001002	Mary Wanjiku (ACTIVE)
3	1001001003	Peter Ochieng (DORMANT)
4	1001001004	Grace Muthoni (CLOSED)
5	1001001005	James Kiprop (ACTIVE)
6-10	No account	Partial data returned

Bonus B - RabbitMQ Message Queue Integration

Overview

A loan officer or branch system publishes a loan application event to the `loan.applications` queue. WSO2 MI consumes it asynchronously, calls the Dashboard API to get the full customer picture, and routes the result based on account status and eligibility.

Queues

Queue	Purpose
loan.applications	Incoming loan application events
loan.approved	Account ACTIVE and eligible true
loan.rejected	Account not ACTIVE or eligibility failed
loan.deadletter	Malformed or unprocessable messages

Message Flow

```
Producer publishes to loan.applications
        |
MI consumes message
        |
Validate required fields
        |
        +-- Invalid --> loan.deadletter (with reason)
        |
Call Dashboard API
        |
        +-- Customer not found --> loan.rejected
        |
Check: accountStatus = ACTIVE AND eligible = true
        |
        +-- YES --> loan.approved
        +-- NO  --> loan.rejected (with reason)
```

How This Differs From Request/Response APIs

The request/response APIs are synchronous. The caller sends a request and waits for a response. If something goes wrong the caller gets an error immediately and must decide whether to retry.

The message queue pattern is asynchronous. The caller publishes a message and moves on immediately without waiting. Retries are handled by the broker not the client. If MI is temporarily down the message stays in the queue and is processed when MI recovers. A branch system can accept loan applications even when the processing backend is under maintenance.

Malformed messages are not silently dropped. They are caught at validation and routed to `loan.deadletter` with the exact validation reason, giving operations teams visibility into bad data without losing the original message.

Assumptions

- JSONPlaceholder and the public SOAP calculator are placeholders. In production these would be internal services behind private networks with service-account authentication and SLAs.
- Credentials are hardcoded for local development. Production would use a secrets manager such as HashiCorp Vault with credentials injected at runtime.
- A single APIM application (DefaultApplication) covers all subscriptions. In production each consumer would have its own application with separate credentials and quotas.
- The Jenkins pipeline assumes the Docker daemon is accessible to the Jenkins user.

What I Would Do Differently With More Time

Webhook-driven pipeline on a dedicated server: Currently Jenkins runs locally and is triggered manually. A GitHub webhook firing against a cloud-hosted Jenkins instance would make the pipeline fully automated and not dependent on a developer's machine being on.

Real backend services: JSONPlaceholder has no authentication and no SLA. In production CustomerProxy would point to the bank's actual CRM behind mutual TLS with vault-managed credentials, and LoanEligibility would call a real underwriting engine with its own compliance requirements.

Environment promotion: One environment currently handles everything. A proper model promotes the same CAR artifact from development through staging to production without rebuilding, with environment-specific endpoint URLs injected at deploy time.

Observability: MI logs would ship to ELK or Splunk with structured fields. APIM analytics would track latency and error rates per API, with alerting on defined thresholds.

How to Run Locally

```
git clone https://github.com/Gathuh/Jamii-Savings.git
cd Jamii-Savings
mvn clean install -DskipTests
docker compose up -d
chmod +x scripts/*.sh
./scripts/wait-for-services.sh
./scripts/deploy-apis.sh
```

Test APIs Directly Against MI

```
curl -s http://localhost:8290/accounts/1.0.0/ACC001/balance
curl -s http://localhost:8290/customers/1.0.0/1
curl -s -X POST http://localhost:8290/loans/1.0.0/eligibility \
  -H "Content-Type: application/json" \
  -d '{"customerId":"C123","monthlyIncome":50000,"loanAmount
     ":20000,"loanTermMonths":12}'
curl -s "http://localhost:8290/dashboard/1.0.0/1/1001001001?
   loanAmount=20000"
```

Portals

Portal	URL
Developer Portal	https://localhost:9443/devportal
Publisher	https://localhost:9443/publisher
RabbitMQ Management	http://localhost:15672

Credentials for all portals: admin / admin (APIM), jamii / jamii123 (RabbitMQ).